COALESCE

Coherence-Observant Adaptive Learning for System-wide Cache Efficiency

Harsh Raj

Under the guidance of
Dr. Bheemappa Halavar

B.Tech Project Mid-Term Review

March 5, 2026

Introduction

Modern processors rely on **cache hierarchies** to reduce the latency gap between CPU speed and main memory access.
Cache replacement policies determine which data blocks remain in the cache and which are evicted.
Traditional policies such as **LRU** and **SRRIP** aim to only maximize **cache hit rate**.
In multicore processors, cache misses may trigger **coherence operations**, including **invalidations** and **writebacks** across cores, increasing latency and interconnect traffic.
Recent research explores **machine learning based replacement policies** to improve eviction decisions.

Literature Review and Gaps Identified

Literature Review 1

Multiperspective Reuse Prediction for Cache Replacement by Daniel A. Jiménez, MICRO 2017

Key Ideas

- Uses perceptron predictors to estimate cache block reuse.
- Utilizes program counter and runtime features.
- Enables fast prediction with low hardware overhead.

Gap Identified

- Focuses only on reuse prediction.
- Does not consider coherence state or sharing behavior.

Literature Review 2

Designing a Cost-Effective Cache Replacement Policy Using Machine Learning by Sethumurugan et al., HPCA 2021

Key Idea

- Uses offline reinforcement learning to guide eviction decisions.
- Achieves less than 5KB hardware budget as required by chip manufacturers.
- Uses Belady's Optimal Algorithm as a metric to judge the results.

Gap Identified

- Suffers from **spatial blindness** because the Program Counter is ignored to satisfy the hardware budget constraint.
- Does not incorporate coherence information.

Literature Review 3

Reinforcement Learning-Based Cache Replacement Policies for Multicore Processors Souza & Freitas, IEEE Access 2024

Key Idea

- Applies reinforcement learning to guide cache eviction decisions.
- Uses coherence state and sharer count as inputs.
- Reduces directory traffic compared with baseline policies.

Gap Identified

- Implemented as an extension to existing policies.
- Limited use of program context information.

Literature Review 4

Concurrency-Aware Cache Miss Cost Prediction with Perceptron Learning Wu et al., GLSVLSI 2025

Key Idea

- Introduces CAMP, a perceptron-based miss cost predictor.
- Uses PC, memory address and access traces as features.
- Uses Memory-Level Parallelism to predict the stall-miss.

Gap Identified

- Focuses only on miss cost prediction while ignoring expensive memory writebacks.

Problem Statement

Existing cache replacement policies largely optimize for **cache hit rate or reuse prediction**, assuming that all cache misses have similar performance impact.
However, in multicore systems, the cost of evicting a block depends on:

- **Coherence state**
- **Sharing patterns across cores**
- **Program execution context**

Policies that ignore these factors may evict blocks that trigger expensive coherence operations and increased interconnect traffic.

Objectives

- Design a **coherence-aware cache replacement policy**.
- Develop a **lightweight prediction mechanism**.
- Implement a **trace-driven evaluation framework**.
- Compare against **baseline and state-of-the-art policies**.

Progress Achieved So Far

Simulation Environment

Custom Event-Driven C++ Simulator

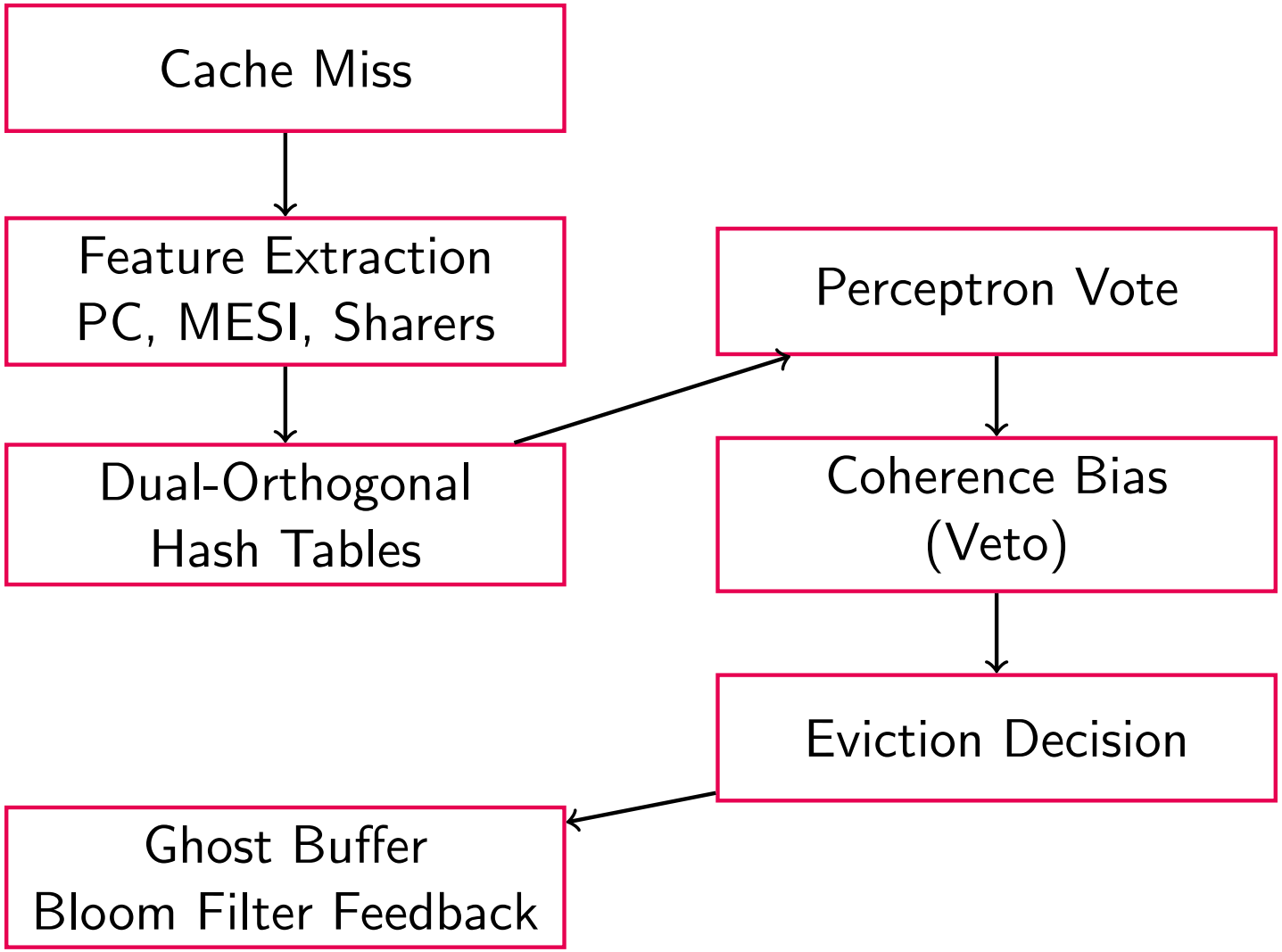
- Target Architecture: 4-core processor
- Shared 8MB Last-Level Cache (16-way set associative)
- Latency Model:
  - Cache Hit = 15 cycles
  - DRAM Miss = 200 cycles
  - Coherence Penalty = +100 cycles
- Baselines: LRU, SRRIP, SHiP, SDBP (State-of-the-Art)

Experimental Scenarios

To evaluate the behavior of the proposed policy under challenging workloads, three stress-test scenarios were designed.

- **Database Scan (Pollution Test)**
  - Large streaming dataset mixed with a small working set.
  - Evaluates the policy's ability to prevent cache pollution.
- **Graph Hub (Coherence Contention)**
  - Frequently shared and modified data across multiple cores.
  - Tests the policy's ability to protect high-cost `MODIFIED` blocks.
- **Phase Change (Adaptation Test)**
  - Workload behavior changes during execution.
  - Evaluates whether the predictor can adapt to new access patterns.

COALESCE Workflow



COALESCE Architecture

- **Dual-Orthogonal Hashing**
  - Two independent hash tables reduce destructive aliasing.
  - Table 0 learns interactions between **PC and coherence state**.
  - Table 1 learns interactions between **PC and sharer count**.
- **Set Sampling (6.25%)**
  - Predictor updates occur only on a subset of cache sets.
  - Reduces hardware overhead and dynamic energy consumption.
- **Coherence Veto Mechanism**
  - Adds bias for blocks in `MODIFIED` or highly shared states.
  - Protects expensive coherence data from premature eviction.

COALESCE Architecture

- **Bloom Filter Ghost Buffer**
  - Tracks recently evicted blocks using a compact Bloom filter.
  - Detects premature evictions and provides feedback to the predictor.
- **Perceptron-Based Predictor**
  - Computes a weighted vote estimating the reuse value of a cache line.
  - Combines multiple architectural features for eviction decisions.
- **Adaptive Threshold Mechanism**
  - Allows strongly negative predictions to override coherence bias.
  - Enables faster adaptation to workload phase changes.

Cost Function and Decision Model

System Objective: True Cost of a Miss

$$Cost(x) = Stall + Traffic + Writeback$$

Perceptron Vote

$$y = \sum_{i=0}^{N} w_i + Bias_{coherence}$$

Coherence Bias (Veto)

$$Bias = \begin{cases} +150 & \text{if State} = \text{MODIFIED} \\ +75 & \text{if Sharers} \geq 2 \\ 0 & \text{otherwise} \end{cases}$$

Simulator Configuration

Architecture

- 4-core system with shared LLC
- 1024-line cache (16-way associative)
- 64B cache line size

Latency Model

- L3 Hit = 15 cycles
- DRAM Access = 200 cycles
- Coherence eviction penalty = +100 cycles

Workload Length

- 10M – 120M memory accesses per scenario

Hardware Overhead

Predictor Storage

- Dual-hashed perceptron tables
- 4096 total weights
- Storage overhead: $<$**5KB**

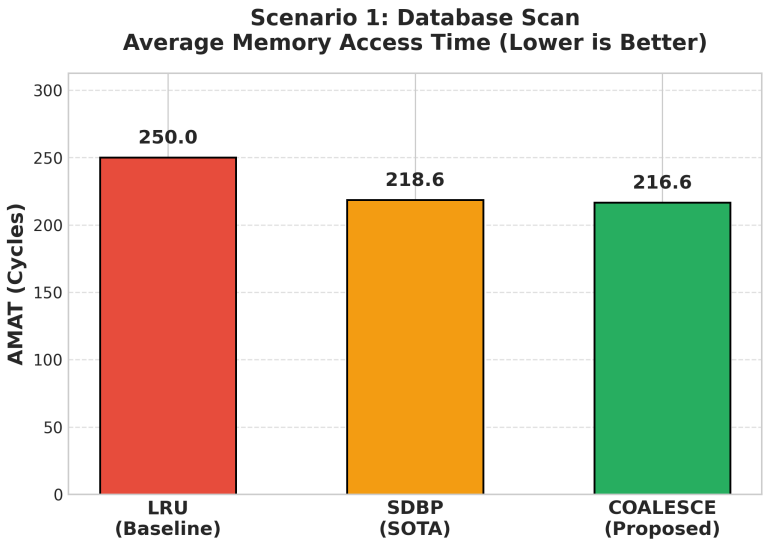
Training Optimization

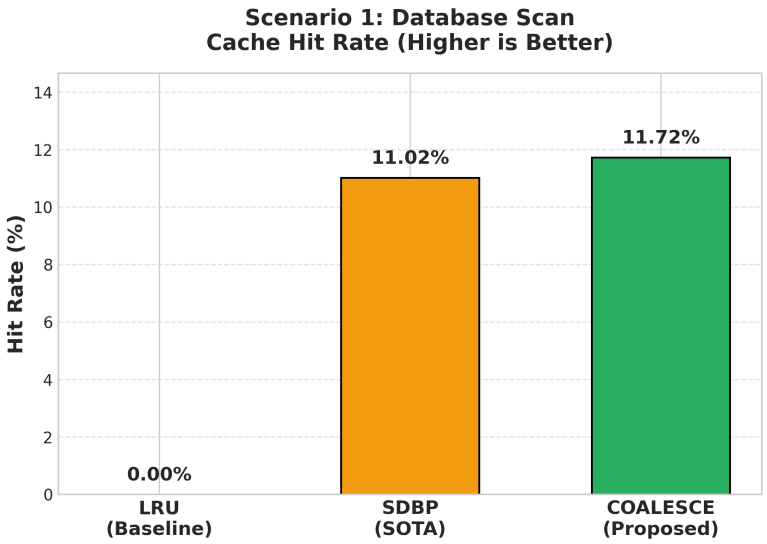
- Set sampling: 6.25% of sets used for training
- Reduces dynamic update energy

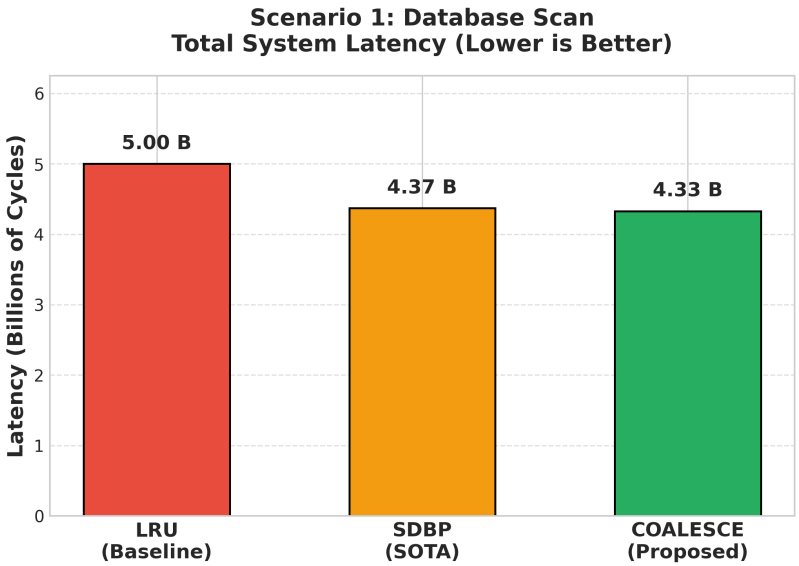
Ghost Buffer

- 1024-bit Bloom filter
- Compact ghost directory (256 entries)
- Detects premature evictions

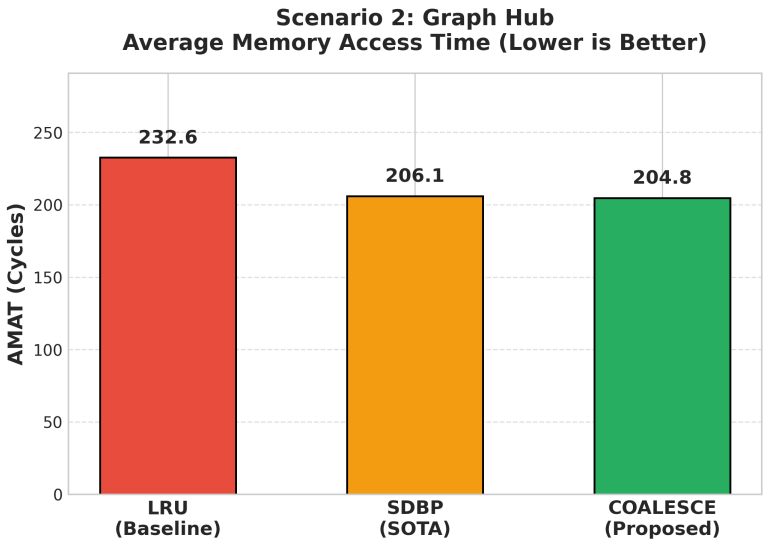
Results: Database Scan (Pollution Test)

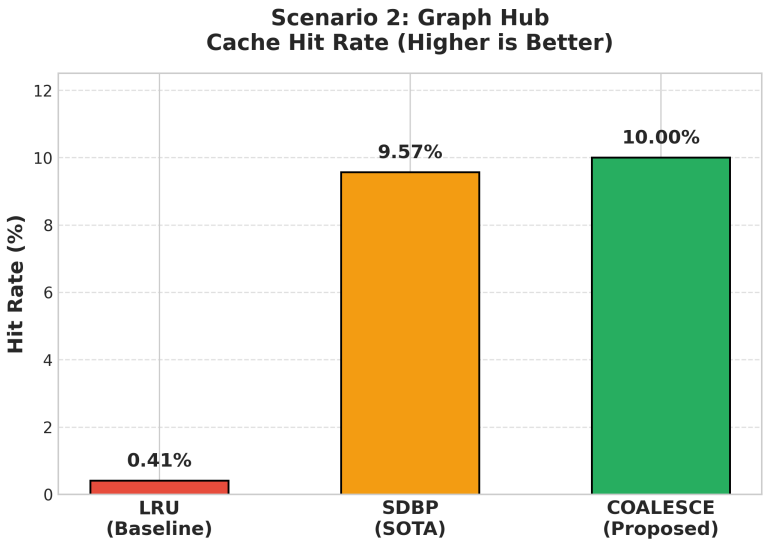


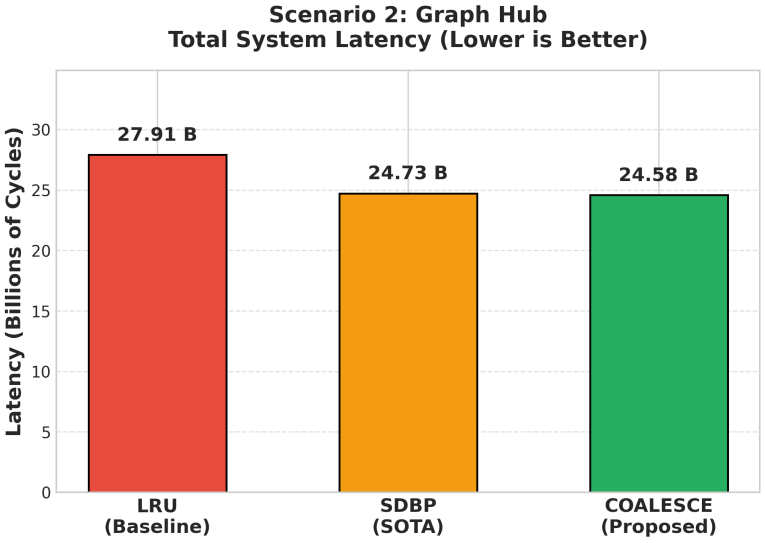




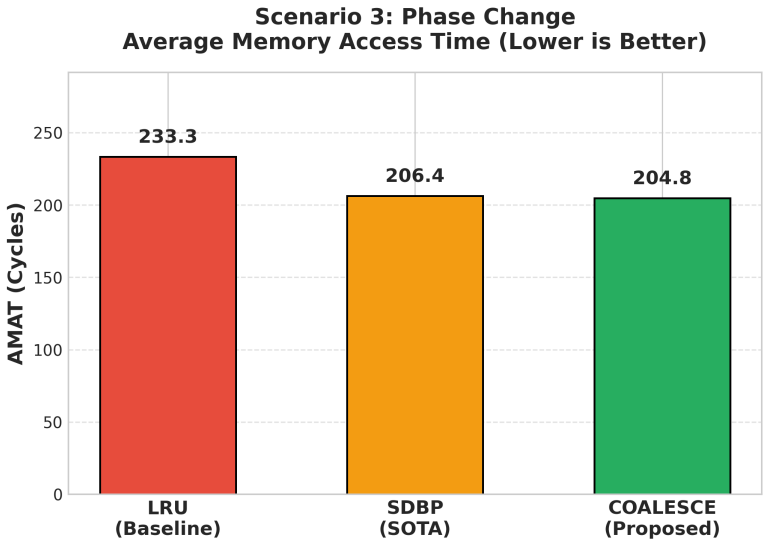
Results: Graph Hub (Coherence Contention)

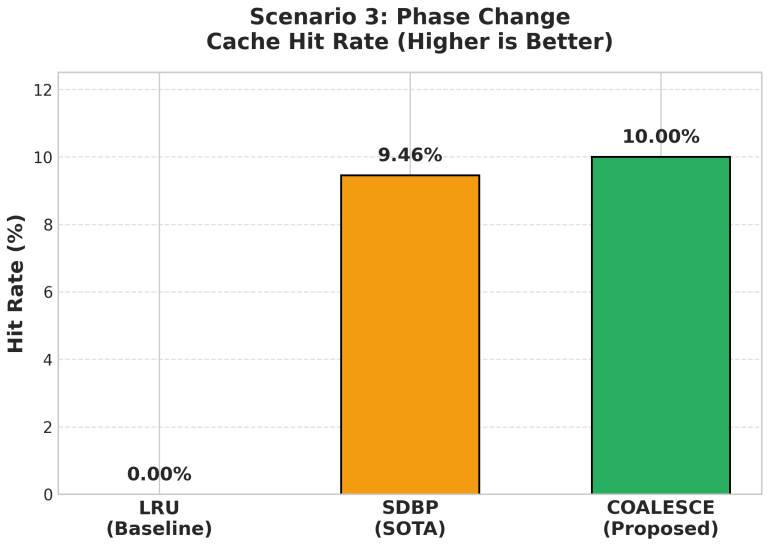


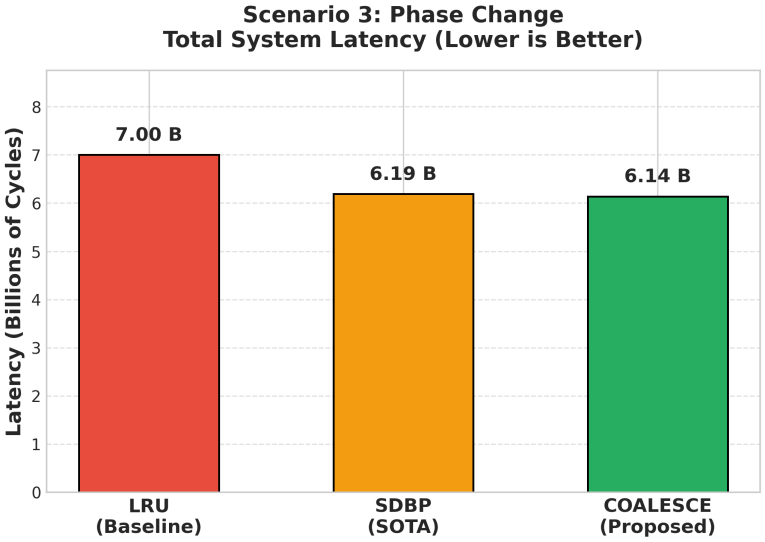




Results: Phase Change (Adaptation)







Performance Summary

COALESCE: Overall Performance Summary

Scenario	Epochs / Iterations	LRU AMAT	SDBP AMAT	COALESCE AMAT	Latency Saved vs SDBP
Database Scan	10 Million Iterations	250.0	218.6	**216.6**	**~40 Million Cycles**
Graph Hub	10 Million Epochs	232.6	206.1	**204.8**	**~149 Million Cycles**
Phase Change	20 Million Iterations	233.3	206.4	**204.8**	**~45 Million Cycles**

Conclusion

- COALESCE integrates **program context, coherence state, and sharing behavior** using a lightweight perceptron predictor.
- COALESCE maintains **hardware overhead below 5KB** using set sampling and compact ghost buffers.
- Experimental results show consistent improvements in **AMAT and hit rate** across pollution, coherence, and phase-change scenarios.

Future Work

- Integrate COALESCE into a **cycle-accurate simulator** (e.g., ChampSim).
- Evaluate performance on **real benchmark suites** such as SPEC CPU and PARSEC.
- Compare COALESCE to **Belady's Optimal Algorithm**.

References

- Jiménez, D. A. Multiperspective Reuse Prediction for Cache Replacement, MICRO 2017.
- Sethumurugan et al. Cost-Effective Cache Replacement Policy Using Machine Learning, HPCA 2021.
- Souza, M. A., Freitas, H. C. RL-Based Cache Replacement Policies for Multicore Processors, IEEE Access 2024.
- Wu, Y. et al. Concurrency-Aware Cache Miss Cost Prediction with Perceptron Learning, GLSVLSI 2025.

Thank You